Chapter 2

AN INTRODUCTION TO TABU SEARCH

Michel Gendreau
Centre de recherche sur les transports
Département d'informatique et de recherche opérationnelle
Université de Montréal
Case postale 6128, Succursale "Centre-ville"
Montréal, Canada H3C 3J7
E-mail: michelg@crt.umontreal.ca

Abstract This chapter presents the fundamental concepts of Tabu Search (TS) in a tutorial fashion. Special emphasis is put on showing the relationships with classical Local Search methods and on the basic elements of any TS heuristic, namely, the definition of the search space, the neighborhood structure, and the search memory. Other sections cover other important concepts such as search intensification and diversification and provide references to significant work on TS. Recent advances in TS are also briefly discussed.

1 INTRODUCTION

Over the last fifteen years, well over a hundred papers presenting applications of Tabu Search (TS), a heuristic method originally proposed by Glover in 1986, to various combinatorial problems have appeared in the operations research literature. In several cases, the methods described provide solutions very close to optimality and are among the most effective, if not the best, to tackle the difficult problems at hand. These successes have made TS extremely popular among those interested in finding good solutions to the large combinatorial problems encountered in many practical settings. Several papers, book chapters, special issues and books have surveyed the rich TS literature (a list of some of the most important references is provided in a later section). In spite of this abundant literature, there still seem to be many researchers who, while they are eager to apply TS to new problem settings, find it difficult to properly grasp the fundamental concepts of the method, its strengths and its limitations, and to come up with effective implementations. The purpose of this chapter is to address this situation by providing an introduction in the form of a tutorial focusing on the fundamental concepts of TS. Throughout the chapter, two relatively straightforward, yet challenging and relevant, problems will be used to illustrate these concepts: the Classical Vehicle Routing Problem (CVRP) and the Capacitated Plant Location Problem (CPLP). These will be introduced in the following section. The remainder of the chapter is organized as follows. The basic concepts of TS (search space, neighborhoods, and short-term tabu lists) are described and illustrated in Section 3. Intermediate, yet critical, concepts, such as intensification and diversification, are described in Section 4. This is followed

in Section 5 by a brief discussion of advanced topics and recent trends in TS, and in Section 6 by a short list of key references on TS and its applications. Section 7 provides practical tips for newcomers struggling with unforeseen problems as they first try to apply TS to their favorite problem. Section 8 concludes the chapter with some general advice on the application of TS to combinatorial problems.

2 ILLUSTRATIVE PROBLEMS

2.1 The Classical Vehicle Routing Problem

Vehicle Routing Problems have very important applications in the area of distribution management. As a consequence, they have become some of the most studied problems in the combinatorial optimization literature and large number of papers and books deal with the numerous procedures that have been proposed to solve them. These include several TS implementations that currently rank among the most effective. The *Classical Vehicle Routing Problem* (CVRP) is the basic variant in that class of problems. It can formally be defined as follows. Let $G = (V, A)$ be a graph where V is the vertex set and A is the arc set. One of the vertices represents the *depot* at which a fleet of m identical vehicles of capacity Q is based, and the other vertices customers that need to be serviced. With each customer vertex v_i are associated a demand q_i and a service time t_i. With each arc (v_i, v_j) of A are associated a cost c_{ij} and a travel time t_{ij}. The CVRP consists in finding a set of routes such that:

1. Each route begins and ends at the depot;
2. Each customer is visited exactly once by exactly one route;
3. The total demand of the customers assigned to each route does not exceed Q;
4. The total duration of each route (including travel and service times) does not exceed a specified value L;
5. The total cost of the routes is minimized.

A feasible solution for the problem thus consists in a partition of the customers into m groups, each of total demand no larger than Q, that are sequenced to yield routes (starting and ending at the depot) of duration no larger than L.

2.2 The Capacitated Plant Location Problem

The *Capacitated Plant Location Problem* (CPLP) is one of the basic problems in Location Theory. It is encountered in many application settings that involve locating facilities with limited capacity to provide services. The CPLP can be formally described as follows. A set of customers I with demands $d_i, i \in I$, for some product are to be served from plants located in a subset of sites from a given set J of "potential sites". For each site $j \in J$, the fixed cost of "opening" the plant at j is f_j and its capacity is K_j. The cost of transporting one unit of the product from site j to customer i is c_{ij}. The objective is to minimize the total cost, i.e., the sum of the fixed costs for open plants and the transportation costs.

Letting x_{ij} $(i \in I, j \in J)$ denote the quantity shipped from site j to customer i (the x_{ij}'s are the so-called *flow variables*) and y_j $(j \in J)$ be a 0–1 variable indicating

whether or not the plant at site j is open (the y_j's are the *location variables*), the problem can be formulated as the following mathematical program:

$$(CPLP) \quad \text{Minimize } z = \sum_{j \in J} f_j y_j + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

$$\text{subject to} \sum_{j \in J} x_{ij} = di, \quad i \in I$$

$$\sum_{i \in I} x_{ij} \leq K_j y_j, \quad j \in J$$

$$x_{ij} \geq 0, \quad i \in I, \quad j \in J$$

$$y_j \in \{0, 1\}, \quad j \in J$$

Remark 2.1. *For any vector $\tilde{\mathbf{y}}$ of location variables, optimal (w.r.t. to this plant configuration) values for the flow variables $\mathbf{x}(\tilde{\mathbf{y}})$ can be retrieved by solving the associated transportation problem:*

$$(TP) \quad \text{Minimize } z(\tilde{\mathbf{y}}) = \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

$$\text{subject to} \sum_{j \in J} x_{ij} = d_i, \quad i \in I$$

$$\sum_{i \in I} x_{ij} \leq K_j \tilde{y}_j, \quad j \in J$$

$$x_{ij} \geq 0, \quad i \in I, \quad j \in J$$

If $\tilde{\mathbf{y}} = \mathbf{y}^*$, the optimal location variable vector, the optimal solution to the original CPLP problem is simply given by $(\mathbf{y}^*, \mathbf{x}(\mathbf{y}^*))$.

Remark 2.2. *An optimal solution of the original CPLP problem can always be found at an extreme point of the polyhedron of feasible flow vectors defined by the constraints:*

$$\sum_{j \in J} x_{ij} = d_i, \quad i \in I$$

$$\sum_{i \in I} x_{ij} \leq K_j, \quad j \in J$$

$$x_{ij} \geq 0, \quad i \in I, \quad j \in J$$

This property follows from the fact that the CPLP can be interpreted as a fixed-charge problem defined in the space of the flow variables. This fixed-charge problem has a concave objective function that always admits an extreme point minimum. The optimal values for the location variables can easily be obtained from the optimal flow vector by setting y_j equal to 1 if $\sum_{i \in I} x_{ij} > 0$, and to 0 otherwise.

To our knowledge, no TS heuristic has ever been proposed for the CPLP, but we will see in the following that this problem can be used to illustrate many important concepts related to the approach.

3 BASIC CONCEPTS

3.1 Historical Background

Before introducing the basic concepts of TS, we believe it is useful to go back in time to try to better understand the genesis of the method and how it relates to previous work.

Heuristics, i.e., approximate solution techniques, have been used since the beginnings of operations research to tackle difficult combinatorial problems. With the development of complexity theory in the early 70's, it became clear that, since most of these problems were indeed *NP-hard,* there was little hope of ever finding efficient exact solution procedures for them. This realization emphasized the role of heuristics for solving the combinatorial problems that were encountered in real-life applications and that needed to be tackled, whether or not they were *NP-hard.* While many different approaches were proposed and experimented with, the most popular one was based on Local Search (LS) improvement techniques. LS can be roughly summarized as an iterative search procedure that, starting from an initial feasible solution, progressively improves it by applying a series of local modifications (or *moves*). At each iteration, the search moves to an improving feasible solution that differs only slightly from the current one (in fact, the difference between the previous and the new solutions amounts to one of the local modifications mentioned above). The search terminates when it encounters a local optimum with respect to the transformations that it considers, an important limitation of the method: unless one is extremely lucky, this local optimum is often a fairly mediocre solution. In LS, the quality of the solution obtained and computing times are usually highly dependent upon the "richness" of the set of transformations (moves) considered at each iteration of the heuristic.

In 1983, the world of combinatorial optimization was shattered by the appearance of a paper in which the authors (Kirkpatrick, Gelatt and Vecchi) were describing a new heuristic approach called *Simulated Annealing* (SA) that could be shown to converge to an optimal solution of a combinatorial problem, albeit in infinite computing time. Based on analogy with statistical mechanics, SA can be interpreted as a form of controlled random walk in the space of feasible solutions. The emergence of SA indicated that one could look for other ways to tackle combinatorial optimization problems and spurred the interest of the research community. In the following years, many other new approaches, mostly based on analogies with natural phenomena, were proposed (TS, *Ant Systems, Threshold Methods)* and, together with some older ones, such as *Genetic Algorithms* (Holland, 1975), they gained an increasing popularity. Now collectively known under the name of *Meta-Heuristics* (a term originally coined by Glover in 1986), these methods have become over the last fifteen years the leading edge of heuristic approaches for solving combinatorial optimization problems.

3.2 Tabu Search

Building upon some of his previous work, Fred Glover proposed in 1986 a new approach, which he called Tabu Search, to allow LS methods to overcome local optima. (In fact, many elements of this first TS proposal, and some elements of later TS elaborations, were introduced in Glover, 1977, including short term memory to prevent the reversal of recent moves, and longer term frequency memory to reinforce attractive components.) The basic principle of TS is to pursue LS whenever it encounters a local optimum by allowing non-improving moves; *cycling* back to previously visited

solutions is prevented by the use of *memories,* called *tabu lists,* that record the recent history of the search, a key idea that can be linked to Artificial Intelligence concepts. It is interesting to note that, the same year, Hansen proposed a similar approach, which he named *steepest ascent/mildest descent.* It is also important to remark that Glover did not see TS as a proper heuristic, but rather as a Meta-Heuristic, i.e., a general strategy for guiding and controlling "inner" heuristics specifically tailored to the problems at hand.

3.3 Search Space and Neighborhood Structure

As we just mentioned, TS is an extension of classical LS methods. In fact, basic TS can be seen as simply the combination of LS with short-term memories. It follows that the two first basic elements of any TS heuristic are the definition of its *search space* and its *neighborhood structure.*

The search space of an LS or TS heuristic is simply the space of all possible solutions that can be considered (visited) during the search. For instance, in the CVRP example described in Section 2, the search space could simply be the set of feasible solutions to the problem, where each point in the search space corresponds to a set of vehicles routes satisfying all the specified constraints. While in that case the definition of the search space seems quite natural, it is not always so. Consider now the CPLP example of Section 2: the feasible space involves both integer location and continuous flow variables that are linked by strict conditions; moreover, as has been indicated before, for any feasible set of values for the location variables, one can fairly easily retrieve optimal values for the flow variables by solving the associated transportation problem. In this context, one could obviously use as a search space the full feasible space; this would involve manipulating both location and flow variables, which is not an easy task. A more attractive search space is the set of feasible vectors of location variables, i.e., feasible vectors in $\{0, 1\}^{|J|}$, any solution in that space being "completed" to yield a feasible solution to the original problem by computing the associated optimal flow variables. It is interesting to note that these two possible definitions are not the only ones. Indeed, on the basis of Remark 2.2, one could also decide to search instead the set of extreme points of the set of feasible flow vectors, retrieving the associated location variables by simply noting that a plant must be open whenever some flow is allocated to it. In fact, this type of approach was used successfully by Crainic, Gendreau and Farvolden (2000) to solve the Fixed Charge Multi-commodity Network Design Problem, a more general problem that includes the CPLP as a special case. It is also important to note that it is not always a good idea to restrict the search space to feasible solutions; in many cases, allowing the search to move to infeasible solutions is desirable, and sometimes necessary (see Section 4.4 for further details).

Closely linked to the definition of the search space is that of the neighborhood structure. At each iteration of LS or TS, the local transformations that can be applied to the current solution, denoted S, define a set of neighboring solutions in the search space, denoted $N(S)$ (the neighborhood of S). Formally, $N(S)$ is a subset of the search space defined by:

$$N(S) = \{\text{solutions obtained by applying a single local transformation to } S\}.$$

In general, for any specific problem at hand, there are many more possible (and even, attractive) neighborhood structures than search space definitions. This follows from the fact that there may be several plausible neighborhood structures for a given definition of the search space. This is easily illustrated on our CVRP example that has been the object of several TS implementations. In order to simplify the discussion, we suppose in the following that the search space is the feasible space.

Simple neighborhood structures for the CVRP involve moving at each iteration a single customer from its current route; the selected customer is inserted in the same route or in another route with sufficient residual capacity. An important feature of these neighborhood structures is the way in which insertions are performed: one could use random insertion or insertion at the best position in the target route; alternately, one could use more complex insertion schemes that involve a partial re-optimization of the target route, such as GENI insertions (see Gendreau et al., 1994). Before proceeding any further it is important to stress that while we say that these neighborhood structures involve moving a *single* customer, the neighborhoods they define contain all the feasible route configurations that can be obtained from the current solution by moving *any* customer and inserting it in the stated fashion. Examining the neighborhood can thus be fairly demanding.

More complex neighborhood structures for the CVRP, such as the λ-interchange of Osman (1993), are obtained by allowing simultaneously the movement of customers to different routes and the swapping of customers between routes. In Rego and Roucairol (1996), moves are defined by *ejection chains* that are sequences of coordinated movements of customers from one route to another; for instance, an ejection chain of length 3 would involve moving a customer v_1 from route R_1 to route R_2, a customer v_2 from R_2 to route R_3 and a customer v_3 from R_3 to route R_4. Other neighborhood structures involve the swapping of sequences of several customers between routes, as in the Cross-exchange of Taillard et al. (1997). These types of neighborhoods have seldom be used for the CVRP, but are common in TS heuristics for its time-windows extension, where customers must be visited within a pre-specified time interval. We refer the interested reader to Gendreau, Laporte and Potvin (2002) and Bräysy and Gendreau (2001) for a more detailed discussion of TS implementations for the CVRP and the Vehicle Routing Problem with Time-Windows.

When different definitions of the search space are considered for any given problem, neighborhood structures will inevitably differ to a considerable degree. This can be illustrated on our CPLP example. If the search space is defined with respect to the location variables, neighborhood structures will usually involve the so-called "*Add/Drop*" and "*Swap*" moves that respectively change the status of one site (i.e., either opening a closed facility or closing an open one) and move an open facility from one site to another (this move amounts to performing simultaneously an Add move and a Drop move). If, however, the search space is the set of extreme points of the set of feasible flow vectors, these moves become meaningless. One should instead consider moves defined by the application of pivots to the linear programming formulation of the transportation problem, since each pivot operation moves the current solution to an adjacent extreme point.

The preceding discussion should have clarified a major point: choosing a search space and a neighborhood structure is by far the most critical step in the design of any TS heuristic. It is at this step that one must make the best use of the understanding and knowledge he/she has of the problem at hand.

3.4 Tabus

Tabus are one of the distinctive elements of TS when compared to LS. As we already mentioned, tabus are used to prevent cycling when moving away from local optima through non-improving moves. The key realization here is that when this situation occurs, something needs to be done to prevent the search from tracing back its steps to where it came from. This is achieved by declaring *tabu* (disallowing) moves that reverse the effect of recent moves. For instance, in the CVRP example, if customer v_1 has just been moved from route R_1 to route R_2, one could declare tabu moving back v_1 from R_2 to R_1 for some number of iterations (this number is called the *tabu tenure* of the move). Tabus are also useful to help the search move away from previously visited portions of the search space and thus perform more extensive exploration.

Tabus are stored in a *short-term memory* of the search (the *tabu list*) and usually only a fixed and fairly limited quantity of information is recorded. In any given context, there are several possibilities regarding the specific information that is recorded. One could record complete solutions, but this requires a lot of storage and makes it expensive to check whether a potential move is tabu or not; it is therefore seldom used. The most commonly used tabus involve recording the last few transformations performed on the current solution and prohibiting reverse transformations (as in the example above); others are based on key characteristics of the solutions themselves or of the moves.

To better understand how tabus work, let us go back to our reference problems. In the CVRP, one could define tabus in several ways. To continue our example where customer v_1 has just been moved from route R_1 to route R_2, one could declare tabu specifically moving back v_1 from R_2 to R_1 and record this in the short-term memory as the triplet (v_1, R_2, R_1). Note that this type of tabu will not constrain the search much, but that cycling may occur if v_1 is then moved to another route R_3 and then from R_3 to R_1. A stronger tabu would involve prohibiting moving back v_1 to R_1 (without consideration for its current route) and be recorded as (v_1, R_1). An even stronger tabu would be to disallow moving v_1 to any other route and would simply be noted as (v_1).

In the CPLP, when searching the space of location variables, tabus on Add/Drop moves should prohibit changing the status of the affected location variable and can be recorded by noting its index; tabus for Swap moves are more complex: they could be declared with respect to the site where the facility was closed, to the site where the facility was opened, to both locations (i.e., changing the status of both location variables is tabu), or to the specific swapping operation. When searching the space of flow variables, one can take advantage of the fact that a pivot operation is associated with a unique pair of entering and leaving variables to define tabus; while here again several combinations are possible, experience has shown that when dealing with pivot neighborhood structures, tabus imposed on leaving variables (to prevent them from coming back in the basis) are usually much more effective.

Multiple tabu lists can be used simultaneously and are sometimes advisable. For instance, in the CPLP, if one uses a neighborhood structure that contains both Add/Drop and Swap moves, it might be a good idea to keep a separate tabu list for each type of moves.

Standard tabu lists are usually implemented as circular lists of fixed length. It has been shown, however, that fixed-length tabus cannot always prevent cycling, and some authors have proposed varying the tabu list length during the search (Glover, 1989, 1990; Skorin-Kapov, 1990; Taillard, 1990, 1991). Another solution is to randomly

generate the tabu tenure of each move within some specified interval; using this approach requires a somewhat different scheme for recording tabus that are then usually stored as *tags* in an array (the entries in this array will usually record the iteration number until which a move is tabu; see Gendreau, Hertz and Laporte, 1994, for more details).

3.5 Aspiration Criteria

While central to TS, tabus are sometimes too powerful: they may prohibit attractive moves, even when there is no danger of cycling, or they may lead to an overall stagnation of the searching process. It is thus necessary to use algorithmic devices that will allow one to *revoke* (cancel) tabus. These are called *aspiration criteria.* The simplest and most commonly used aspiration criterion (found in almost all TS implementations) consists in allowing a move, even if it is tabu, if it results in a solution with an objective value better than that of the current best-known solution (since the new solution has obviously not been previously visited). Much more complicated aspiration criteria have been proposed and successfully implemented (see, for instance, de Werra and Hertz, 1989, or Hertz and de Werra, 1991), but they are rarely used. The key rule in this respect is that if cycling cannot occur, tabus can be disregarded.

3.6 A Template for Simple Tabu Search

We are now in the position to give a general template for TS, integrating the elements we have seen so far. We suppose that we are trying to minimize a function $f(S)$ over some domain and we apply the so-called "best improvement" version of TS, i.e., the version in which one chooses at each iteration the best available move (this is the most commonly used version of TS).

Notation

- S, the current solution,
- S^*, the best-known solution,
- f^*, value of S^*,
- $N(S)$, the neighborhood of S,
- $\tilde{N}(S)$, the "admissible" subset of $N(S)$ (i.e., non-tabu or allowed by aspiration).

Initialization

Choose (construct) an initial solution S_0.
Set $S := S_0$, $f^* := f(S_0)$, $S^* := S_0$, $T := \emptyset$.

Search

While *termination criterion not satisfied* do

- Select S in argmin $[f(S')]$; $S' \varepsilon \tilde{N}(S)$
- if $f(S) < f^*$, then set $f^* := f(S)$, $S^* := S$;
- record tabu for the current move in T (delete oldest entry if necessary);

endwhile.

3.7 Termination Criteria

One may have noticed that we have not specified in our template above a termination criterion. In theory, the search could go on forever, unless the optimal value of the problem at hand is known beforehand. In practice, obviously, the search has to be stopped at some point. The most commonly used stopping criteria in TS are:

- after a fixed number of iterations (or a fixed amount of CPU time);
- after some number of iterations without an improvement in the objective function value (the criterion used in most implementations);
- when the objective reaches a pre-specified threshold value.

In complex tabu schemes, the search is usually stopped after completing a sequence of *phases,* the duration of each phase being determined by one of the above criteria.

3.8 Probabilistic TS and Candidate Lists

In "regular" TS, one must evaluate the objective for every element of the neighborhood $N(S)$ of the current solution. This can prove extremely expensive from the computational standpoint. An alternative is to instead consider only a random sample $N'(S)$ of $N(S)$, thus reducing significantly the computational burden. Another attractive feature of this alternative is that the added randomness can act as an anti-cycling mechanism; this allows one to use shorter tabu lists than would be necessary if a full exploration of the neighborhood was performed. One the negative side, it must be noted that, in that case, one may miss excellent solutions (more on this topic in Section 7.3). Probabilities may also be applied to activating tabu criteria.

Another way to control the number of moves examined is by means of *candidate list* strategies, which provide more strategic ways of generating a useful subset $N'(S)$ of $N(S)$. (The probabilistic approach can be considered to be one instance of a candidate list strategy, and may also be used to modify such a strategy.) Failure to adequately address the issues involved in creating effective candidate lists is one of the more conspicuous shortcomings that differentiates a naive TS implementation from one that is more solidly grounded. Relevant designs for candidate list strategies are discussed in Glover and Laguna (1997). We also discuss a useful type of candidate generation approach in Section 4.4.

4 INTERMEDIATE CONCEPTS

Simple TS as described above can sometimes successfully solve difficult problems, but in most cases, additional elements have to be included in the search strategy to make it fully effective. We now briefly review the most important of these.

4.1 Intensification

The idea behind the concept of search intensification is that, as an intelligent human being would probably do, one should explore more thoroughly the portions of the search space that seem "promising" in order to make sure that the best solutions in these areas are indeed found. From time to time, one would thus stop the normal searching process to perform an intensification phase. In general, intensification is based on some

intermediate-term memory, such as a *recency memory,* in which one records the number of consecutive iterations that various "solution components" have been present in the current solution without interruption. For instance, in the CPLP application, one could record how long each site has had an open facility. A typical approach to intensification is to restart the search from the best currently known solution and to "freeze" (fix) in it the components that seem more attractive. To continue the CPLP example, one could thus freeze a number of facilities in the sites that have had them for the largest number of iterations and perform a restricted search on the other sites. Another technique that is often used consists in changing the neighborhood structure to one allowing more powerful or more diverse moves. In the CVRP example, one could therefore allow more complex insertion moves or switch to an ejection chain neighborhood structure. In the CPLP example, if Add/Drop moves were used, Swap moves could be added to the neighborhood structure. In probabilistic TS, one could increase the sample size or switch to searching without sampling.

Intensification is used in many TS implementations, but it is not always necessary. This is because there are many situations where the search performed by the normal searching process is thorough enough. There is thus no need to spend time exploring more carefully the portions of the search space that have already been visited, and this time can be used more effectively as we shall see right now.

4.2 Diversification

One of the main problems of all methods based on Local Search approaches, and this includes TS in spite of the beneficial impact of tabus, is that they tend to be too "local" (as their name implies), i.e., they tend to spend most, if not all, of their time in a restricted portion of the search space. The negative consequence of this fact is that, although good solutions may be obtained, one may fail to explore the most interesting parts of the search space and thus end up with solutions that are still pretty far from the optimal ones. *Diversification* is an algorithmic mechanism that tries to alleviate this problem by forcing the search into previously unexplored areas of the search space. It is usually based on some form of *long-term memory* of the search, such as a *frequency memory,* in which one records the total number of iterations (since the beginning of the search) that various "solution components" have been present in the current solution or have been involved in the selected moves. For instance, in the CPLP application, one could record during the number of iterations during which each site has had an open facility. In the CVRP application, one could note how many times each customer has been moved from its current route. In cases where it is possible to identify useful "regions" of the search space, the frequency memory can be refined to track the number of iterations spent in these different regions.

There are two major diversification techniques. The first, called *restart diversification,* involves forcing a few rarely used components in the current solution (or the best known solution) and restarting the search from this point. In CPLP procedures, one could thus open one or a few facilities at locations that have seldom had them up to that point and resume searching from that plant configuration (one could also close facilities at locations that have been used the most frequently). In a CVRP heuristic, customers that have not yet been moved frequently could be forced into new routes. The second diversification method, *continuous diversification,* integrates diversification considerations directly into the regular searching process. This is achieved by *biasing* the

evaluation of possible moves by adding to the objective a small term related to component frequencies (see Soriano and Gendreau, 1996, for an extensive discussion on these two techniques). A third way of achieving diversification is *strategic oscillation* as we will see in the next subsection.

Before closing this subsection, we would like to stress that ensuring proper search diversification is possibly **the most critical issue** in the design of TS heuristics. It should be addressed with extreme care fairly early in the design phase and revisited if the results obtained are not up to expectations.

4.3 Allowing Infeasible Solutions

Accounting for all problem constraints in the definition of the search space often restricts the searching process too much and can lead to mediocre solutions. This occurs, for example, in CVRP instances where the route capacity or duration constraints are too tight to allow moving customers effectively between routes. In such cases, *constraint relaxation* is an attractive strategy, since it creates a larger search space that can be explored with "simpler" neighborhood structures. Constraint relaxation is easily implemented by dropping selected constraints from the search space definition and adding to the objective weighted penalties for constraint violations. This, however, raises the issue of finding correct weights for constraint violations. An interesting way of circumventing this problem is to use *self-adjusting penalties,* i.e., weights are adjusted dynamically on the basis of the recent history of the search: weights are increased if only infeasible solutions were encountered in the last few iterations, and decreased if all recent solutions were feasible (see, for instance, Gendreau, Hertz and Laporte, 1994, for further details). Penalty weights can also be modified systematically to drive the search to cross the feasibility boundary of the search space and thus induce diversification. This technique, known as *strategic oscillation,* was introduced as early as 1977 by Glover and used since in several successful TS procedures. (An important early variant oscillates among alternative types of moves, hence neighborhood structures, while another oscillates around a selected value for a critical function.)

4.4 Surrogate and Auxiliary Objectives

There are many problems for which the true objective function is quite costly to evaluate, a typical example being the CPLP when one searches the space of location variables. (Remember that, in this case, computing the objective value for any potential solution entails solving the associated transportation problem.) When this occurs, the evaluation of moves may become prohibitive, even if sampling is used. An effective approach to handle this issue is to evaluate neighbors using a *surrogate objective,* i.e., a function that is correlated to the true objective, but is less computationally demanding, in order to identify a (small) set of promising candidates (potential solutions achieving the best values for the surrogate). The true objective is then computed for this small set of candidate moves and the best one selected to become the new current solution. (See Crainic et al., 1993, for an example of this approach.)

Another frequently encountered difficulty is that the objective function may not provide enough information to effectively drive the search to more interesting areas of the search space. A typical illustration of this situation is the variant of the CVRP in which the fleet size is not fixed, but is rather the primary objective (i.e., one is looking

for the minimal fleet size allowing a feasible solution). In this problem, except for solutions where a route has only one or a few customers assigned to it, most neighborhood structures will lead to the situation where all elements in the neighborhood score equally with respect to the primary objective (i.e., all allowable moves produce solutions with the same number of vehicles). In such a case, it is absolutely necessary to define an *auxiliary objective function* to orient the search. Such a function must measure in some way the desirable attributes of solutions. In our example, one could, for instance, use a function that would favor solutions with routes having just a few customers, thus increasing the likelihood that a route can be totally emptied in a subsequent iteration. It should be noted that coming up with an effective auxiliary objective is not always easy and may require a lengthy trial and error process. In some other cases, fortunately, the auxiliary objective is obvious for anyone familiar with the problem at hand. (See Gendreau, Soriano and Salvail, 1993, for an illustration.)

5 ADVANCED TOPICS AND RECENT TRENDS IN TABU SEARCH

The concepts and techniques described in the previous sections are sufficient to design effective TS heuristics for many combinatorial problems. Most early TS implementations, several of which were extremely successful, relied indeed almost exclusively on these algorithmic components. Nowadays, however, most leading edge research in TS makes use of more advanced concepts and techniques. While it is clearly beyond the scope of an introductory tutorial, such as this one, to review this type of advanced material, we would like to give readers some insight into it by briefly describing some current trends in TS research. Readers who wish to learn more about this topic should read our survey paper (Gendreau, 2002) and some of the references provided in the next section.

A large part of the recent research in TS deals with various techniques for making the search more effective. These include methods for exploiting better the information that becomes available during search and creating better starting points, as well as more powerful neighborhood operators and parallel search strategies. (On this last topic, see the taxonomy of Crainic, Toulouse and Gendreau, 1997, and the survey of Cung et al., 2002.) The numerous techniques for making better use of the information are of particular significance since they can lead to dramatic performance improvements. Many of these rely on *elite solutions* (the best solutions previously encountered) or on parts of these to create new solutions, the rationale being that "fragments" (elements) of excellent solutions are often identified quite early in the searching process, but that the challenge is to complete these fragments or to recombine them (Glover, 1992; Glover and Laguna, 1993, 1997; Rochat and Taillard, 1995). Other methods, such as the *Reactive Tabu Search* of Battiti and Tecchiolli (1994), attempt to find ways of making the search move away from local optima that have already been visited.

Another important trend in TS (this is, in fact, a pervasive trend in the whole meta-heuristics field) is *hybridization,* i.e., using TS in conjunction with other solution approaches such as Genetic Algorithms (Fleurent and Ferland, 1996; Crainic and Gendreau, 1999), Lagrangean relaxation (Grünert, 2002), Constraint Programming (Pesant and Gendreau, 1999), column generation (Crainic et al., 2000) and integer

programming techniques (there is a whole chapter on this topic in Glover and Laguna, 1997).

TS research has also started moving away from its traditional application areas (graph theory problems, scheduling, vehicle routing) to new ones: continuous optimization (Rolland, 1996), multi-criteria optimization, stochastic programming, mixed integer programming (Lokketangen and Glover, 1996; Crainic et al., 2000), real-time decision problems (Gendreau et al., 1999), etc. These new areas confront researchers with new challenges that, in turn, call for novel and original extensions of the method.

6 KEY REFERENCES

Readers who wish to read other introductory papers on TS can choose among several ones. Let us mention the ones by de Werra and Hertz (1989), Hertz and de Werra (1991), Glover and Laguna (1993), Glover, Taillard and de Werra (1993) and, in French, by Soriano and Gendreau (1997). The book by Glover and Laguna (1997) is the ultimate reference on TS: apart from the fundamental concepts of the method, it presents a considerable amount of advanced material, as well as a variety of applications. It is interesting to note that this book contains several ideas applicable to TS that yet remain to be fully exploited. The issues of *Annals of Operations Research,* respectively devoted to "Tabu Search" (Glover et al., 1993) and "Metaheuristics in Combinatorial Optimization" (Laporte and Osman, 1996), and the books made up from selected papers presented at the Meta-Heuristics International Conferences (MIC) are also extremely valuable. At this time, the books for the 1995 Breckenridge conference (Osman and Kelly, 1996), the 1997 Sophia-Antipolis one (Voss et al., 1999) and the 1999 Angra dos Reis one (Ribeiro and Hansen, 2002) have been published. The proceedings of MIC'2001, held in Porto, are currently available electronically on the website located at URL:http://tew.ruca.ua.ac.be/eume/MIC2001.

7 TRICKS OF THE TRADE

7.1 Getting Started

Newcomers to TS trying to apply the method to a problem that they wish to solve are often confused about what they need to do to come up with a successful implementation. Basically, they do not know where to start. We believe that the following step-by-step procedure can help a lot and provides a useful framework for getting started.

A step-by-step procedure

1. **Read** one or two good introductory papers to gain some knowledge of the concepts and of the vocabulary.
2. **Read** several papers describing in detail applications in various areas to see how the concepts have been actually implemented by other researchers.
3. **Think** a lot about the problem at hand, focusing on the definition of the **search space** and the **neighborhood structure**.
4. **Implement** a **simple** version based on this search space definition and this neighborhood structure.

5. **Collect statistics** on the performance of this simple heuristic. It is usually useful at this point to introduce a variety of **memories**, such as frequency and recency memories, to really track down what the heuristic does.
6. **Analyze results** and **adjust** the procedure accordingly. It is at this point that one should eventually introduce mechanisms for search intensification and diversification or other intermediate features. Special attention should be paid to **diversification**, since this is often where simple TS procedures fail.

7.2 More Tips

It is not unusual that, in spite of following carefully the preceding procedure, one ends up with a heuristic that nonetheless produces mediocre results. If this occurs, the following tips may prove useful:

1. If there are **constraints**, consider **penalizing** them. Letting the search move to infeasible solutions is often necessary in highly constrained problems to allow for a meaningful exploration of the search space (see Section 4).
2. Reconsider the **neighborhood structure** and change it if necessary. Many TS implementations fail because the neighborhood structure is too simple. In particular, one should make sure that the chosen neighborhood structure allows for a purposeful evaluation of possible moves (i.e., the moves that seem intuitively to move the search in the "right" direction should be the ones that are likely to be selected); it might also be a good idea to introduce a **surrogate objective** (see Section 4) to achieve this.
3. **Collect more statistics.**
4. **Follow the execution of the algorithm step-by-step** on some reasonably sized instances.
5. Reconsider **diversification**. As mentioned earlier, this is a critical feature in most TS implementations.
6. **Experiment** with **parameter** settings. Many TS procedures are extremely sensitive to parameter settings; it is not unusual to see the performance of a procedure dramatically improve after changing the value of one or two key parameters (unfortunately, it is not always obvious to determine which parameters are the key ones in a given procedure).

7.3 Additional Tips for Probabilistic TS

While it is an effective way of tackling many problems, probabilistic TS creates problems of its own that need to be carefully addressed. The most important of these is the fact that, more often than not, the best solutions returned by probabilistic TS will not be local optima with respect to the neighborhood structure being used. This is particularly annoying since, in that case, better solutions can be easily obtained, sometimes even manually. An easy way to come around this is to simply perform a local improvement phase (using the same neighborhood operator) from the best found solution at the end of the TS itself. One could alternately switch to TS without sampling (again from the best found solution) for a short duration before completing the algorithm. A possibly more effective technique is to add throughout the search an intensification step without sampling; in this fashion, the best solutions available in the various regions of the

search space explored by the method will be found and recorded. (Glover and Laguna, 1993, similarly proposed special aspiration criteria for allowing the search to reach local optima at useful junctures.)

7.4 Parameter Calibration and Computational Testing

Parameter calibration and computational experiments are key steps in the development of any algorithm. This is particularly true in the case of TS, since the number of parameters required by most implementations is fairly large and since the performance of a given procedure can vary quite significantly when parameter values are modified. The first step in any serious computational experimentation is to select a good set of benchmark instances (either by obtaining them from other researchers or by constructing them), preferably with some reasonable measure of their difficulty and with a wide range of size and difficulty. This set should be split into two subsets, the first one being used at the algorithmic design and parameter calibration steps, and the second reserved for performing the final computational tests that will be reported in the paper(s) describing the heuristic under development. The reason for doing so is quite simple: when calibrating parameters, one always run the risk of *overfitting,* i.e., finding parameter values that are excellent for the instances at hand, but poor in general, because these values provide too good a "fit" (from the algorithmic standpoint) to these instances. Methods with several parameters should thus be calibrated on much larger sets of instances than ones with few parameters to ensure a reasonable degree of robustness. The calibration process itself should proceed in several stages:

1. Perform exploratory testing to find good ranges of parameters. This can be done by running the heuristic with a variety of parameter settings.
2. Fix the value of parameters that appear to be "robust", i.e., which do not seem to have a significant impact on the performance of the procedure.
3. Perform systematic testing for the other parameters. It is usually more efficient to test values for only a single parameter at a time, the others being fixed at what appear to be reasonable values. One must be careful, however, for *cross effects* between parameters. Where such effects exist, it can be important to jointly test pairs or triplets of parameters, which can be an extremely time-consuming task.

The paper by Crainic et al. (1993) provides a detailed description of the calibration process for a fairly complex TS procedure and can used as a guideline for this purpose.

8 CONCLUSION

Tabu Search is a powerful algorithmic approach that has been applied with great success to many difficult combinatorial problems. A particularly nice feature of TS is that, like all approaches based on Local Search, it can quite easily handle the "dirty" complicating constraints that are typically found in real-life applications. It is thus a really practical approach. It is not, however, a panacea: every reviewer or editor of a scientific journal has seen more than his/her share of failed TS heuristics. These failures stem from two major causes: an insufficient understanding of fundamental concepts of the method (and we hope that this tutorial will help in alleviating this shortcoming), but also, more often than not, a crippling lack of understanding of the problem at hand. One cannot develop

a good TS heuristic for a problem that he/she does not know well! This is because significant problem knowledge is absolutely required to perform the most basic steps of the development of any TS procedure, namely the choice of a search space and of an effective neighborhood structure. If the search space and/or the neighborhood structure are inadequate, no amount of TS expertise will be sufficient to save the day. A last word of caution: to be successful, all meta-heuristics need to achieve both *depth* and *breadth* in their searching process; depth is usually not a problem for TS, which is quite aggressive in this respect (TS heuristics generally find pretty good solutions very early in the search), but breadth can be a critical issue. To handle this, it is extremely important to develop an effective diversification scheme.

ACKNOWLEDGEMENTS

The author is grateful to the Canadian Natural Sciences and Engineering Council and the Fonds FCAR of the Province of Quebec for their financial support. The author also wishes to thank Fred Glover for his insightful comments on an earlier version of this chapter.

REFERENCES

Battiti, R. and Tecchiolli, G. (1994) The reactive tabu search. *ORSA Journal on Computing,* **6**, 126–140.

Bräysy, O. and Gendreau, M. (2001) Tabu search heuristics for the vehicle routing problem with time windows. Report STF42 A01022, SINTEF Applied Mathematics, Oslo, Norway. Forthcoming in *TOP.*

Crainic, T.G. and Gendreau, M. (1999). Towards an evolutionary method—cooperative multi-thread parallel tabu search heuristic Hybrid. In S. Voss, S. Martello, I.H. Osman and C. Roucairol (Eds.), *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization,* Kluwer Academic Publishers, pp. 331–344.

Crainic, T.G., Gendreau, M., and Farvolden, J.M. (2000) Simplex-based tabu search for the multicommodity capacitated fixed charge network design problem. *INFORMS Journal on Computing,* **12**, 223–236.

Crainic, T.G., Gendreau, M., Soriano, P., and Toulouse, M. (1993) A tabu search procedure for multicommodity location/allocation with balancing requirements. *Annals of Operations Research,* **41**, 359–383.

Crainic, T.G., Toulouse, M., and Gendreau, M. (1997) Toward a taxonomy of parallel tabu search heuristics. *INFORMS Journal on Computing,* **9**, 61–72.

Cung, V.-D., Martins, S.L., Ribeiro, C.C., and Roucairol, C. (2002) Strategies for the parallel implementation of metaheuristics. In C.C. Ribeiro and P. Hansen (Eds.), *Essays and Surveys in Metaheuristics,* Kluwer Academic Publishers, pp. 263–308.

Fleurent, C. and Ferland, J.A. (1996) Genetic and hybrid algorithms for graph colouring. *Annals of Operations Research,* **63**, 437–461.

Gendreau, M. (2002) Recent advances in tabu search. In C.C. Ribeiro and P. Hansen (eds.), *Essays and Surveys in Metaheuristics,* Kluwer Academic Publishers, pp. 369–377.

Gendreau, M., Guertin, F., Potvin, J.-Y. and Taillard, É.D. (1999) Paralle tabu search for real-time vehicle routing and dispatching. *Transportation Science,* **33**, 381-390.

Gendreau, M., Hertz, A., and Laporte, G. (1994) A tabu search heuristic for the vehicle routing problem. *Management Science,* **40**, 1276–1290.

Gendreau, M., Laporte, G. and Potvin, J.-Y. (2002) Metaheuristics for the capacitated VRP. In P. Toth and D. Vigo (Eds.), *The Vehicle Routing Problem,* SIAM Monographs on Discrete Mathematics and Applications, pp. 129-154.

Gendreau, M., Soriano, P. and Salvail, L. (1993) Solving the maximum clique problem using a tabu search approach. *Annals of Operations Research,* **41**, 385–403.

Glover, F. (1977) Heuristics for integer programming using surrogate constraints. *Decision Sciences,* **8**, 156–166.

Glover, F. (1986) Future paths for integer programming and links to artificial intelligence. *Computers and Operations Research,* **13**, 533–549.

Glover, F. (1989) Tabu Search—Part I. *ORSA Journal on Computing,* **1**, 190–206.

Glover, F. (1990) Tabu Search—Part II. *ORSA Journal on Computing,* **2**, 4–32.

Glover, F. (1992) Ejection chains, reference structures and alternating path methods for traveling salesman problems. University of Colorado. Shortened version published in *Discrete Applied Mathematics,* **65**, 223–253, 1996.

Glover, F. and Laguna, M. (1993) Tabu search. In C.R. Reeves (Ed.), *Modern Heuristic Techniques for Combinatorial Problems,* Blackwell, pp. 70–150.

Glover, F. and Laguna, M. (1997) *Tabu Search,* Kluwer Academic Publishers, Norwell, MA.

Glover, F., Laguna, M., Taillard, É., and de Werra, D. (Eds.) (1993) Tabu search. *Annals of Operations Research,* **41**, J.C. Baltzer Science Publishers, Basel, Switzerland.

Glover, F., Taillard, É. and de Werra, D. (1993) A user's guide to tabu search. *Annals of Operations Research,* **41**, 3–28.

Grünert, T. (2002) Lagrangean tabu search. In C.C. Ribeiro and P. Hansen (Eds.) *Essays and Surveys in Metaheuristics,* Kluwer Academic Publishers, pp. 379–397.

Hertz, A. and de Werra, D. (1991) The tabu search metaheuristic: how we used it. *Annals of Mathematics and Artificial Intelligence,* **1**, 111–121.

Holland, J.H. (1975) *Adaptation in Natural and Artificial Systems,* University of Michigan Press, Ann Arbor, MI.

Kirkpatrick, S., Gelatt Jr., C.D., and Vecchi, M.P. (1983) Optimization by simulated annealing. *Science,* **220,** 671–680.

Laporte, G. and Osman, I.H. (Eds.) (1996) Metaheuristics in combinatorial optimization. *Annals of Operations Research,* **63**, J.C. Baltzer Science Publishers, Basel, Switzerland.

Lokketangen, A. and Glover, F. (1996) Probabilistic move selection in tabu search for 0/1 mixed integer programming problems. In I.H. Osman and J.P. Kelly (Eds.), *Meta-Heuristics: Theory and Applications,* Kluwer Academic Publishers, pp. 467–488.

Osman, I.H. (1993) Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem. *Annals of Operations Research,* **41**, 421–451.

Osman, I.H. and Kelly, J.P. (Eds.) (1996) *Meta-Heuristics: Theory and Applications,* Kluwer Academic Publishers, Norwell, MA.

Pesant, G. and Gendreau, M. (1999) A constraint programming framework for local search methods. *Journal of Heuristics,* **5**, 255–280.

Rego, C. and Roucairol, C. (1996) A parallel tabu search algorithm using ejection chains for the vehicle routing problem. In I.H. Osman and J.P. Kelly (Eds.), *Meta-Heuristics: Theory and Applications,* Kluwer Academic Publishers, pp. 661–675.

Ribeiro, C.C. and Hansen, P. (Eds.) (2002) *Essays and Surveys in Metaheuristics,* Kluwer Academic Publishers, Norwell, MA.

Rochat, Y. and Taillard, É.D. (1995) Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics,* **1**, 147–167.

Rolland, E. (1996) A tabu search method for constrained real-number search: applications to portfolio selection. Working Paper, The Gary Anderson Graduate School of Management, University of California, Riverside.

Skorin-Kapov, J. (1990) Tabu search applied to the quadratic assignment problem. *ORSA Journal on Computing,* **2**, 33–45.

Soriano, P. and Gendreau, M. (1996) Diversification strategies in tabu search algorithms for the maximum clique problems. *Annals of Operations Research,* **63**, 189–207.

Soriano, P. and Gendreau, M. (1997) Fondements et applications des méthodes de recherche avec tabous. *RAIRO (Recherche opérationnelle),* **31**, 133–159 (in French).

Taillard, É. (1990) Some efficient heuristic methods for the flow shop sequencing problem. *European Journal of Operational Research,* **47**, 65–74.

Taillard, É. (1991) Robust taboo search for the quadratic assignment problem. *Parallel Computing,* **17**, 443–455.

Taillard, É.D., Badeau, P., Gendreau, M. Guertin, F. and Potvin, J.-Y. (1997) A tabu search heuristic for the vehicle routing problem with soft time windows. *Transportation Science,* **31**, 170–186.

Voss, S., Martello, S., Osman, I.H. and Roucairol, C. (Eds.) (1999) *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization,* Kluwer Academic Publishers, Norwell, MA.

de Werra, D. and Hertz, A. (1989) Tabu search techniques: a tutorial and an application to neural networks. *OR Spektrum,* **11**, 131–141.